```
import numpy as np
from collections import defaultdict

class SimpleGridworld:
    def __init__(self):
        self.grid_size = 4
        self.n_states = 16
        self.terminal_states = {0, 15}

    def reset(self):
        self.state = np.random.choice(range(1, 14))
        return self.state

    def step(self, action):
        if self.state in self.terminal_states:
            return self.state, 0, True
        row, col = divmod(self.state, self.grid_size)
        if action == 0: row = max(0, row - 1)
        elif action == 1: row = min(self.grid_size - 1, row + 1)
        elif action == 2: col = max(0, col - 1)
        elif action == 3: col = min(self.grid_size - 1, col + 1)
        self.state = row * self.grid_size + col
        return self.state, -1, self.state in self.terminal_states

def first_visit_mc_prediction(env, num_episodes, gamma=1.0):
    returns_sum = defaultdict(float)
    returns_count = defaultdict(float)
    V = np.zeros(env.n_states)

    for _ in range(num_episodes):
        episode = []
        state = env.reset()
        done = False
        while not done:
            action = np.random.choice(4)
            next_state, reward, done = env.step(action)
            episode.append((state, reward))
            state = next_state

        G = 0
        states_in_episode = [step[0] for step in episode]
        for t in range(len(episode) - 1, -1, -1):
            s, r = episode[t]
            G = gamma * G + r
            if s not in states_in_episode[:t]:
                returns_sum[s] += G
                returns_count[s] += 1
                V[s] = returns_sum[s] / returns_count[s]
    return V
```

```python
if __name__ == "__main__":
    env = SimpleGridworld()
    state_values = first_visit_mc_prediction(env, 5000)
    print(state_values.reshape(4, 4).round(2))
```

```
[[  0.   -13.66 -19.84 -22.02]
 [-13.72 -18.18 -19.92 -20.15]
 [-19.98 -19.69 -17.66 -13.92]
 [-22.11 -20.07 -13.9    0.  ]]
```